



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

**РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:**

Студент _____
(Группа)

(Подпись, дата)

(И.О. Фамилия)

Руководитель НИР

(Подпись, дата)

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Архитектура СУБД	6
1.1 Процесс исполнения SQL-запроса	6
2 Реляционная алгебра	9
2.1 Основные операторы реляционной алгебры	9
2.2 Преобразование синтаксического дерева к реляционной алгебре	10
3 Цель оптимизатора	11
4 Обзор архитектур оптимизаторов	16
4.1 System R	16
4.2 Volcano	17
5 Архитектура Cascades	19
5.1 Группы эквивалентности и выражения	19
5.2 Структура Мемо	20
5.3 Алгоритм поиска	22
6 Правила трансформации и реализации	24
6.1 Правила трансформации	24
6.2 Правила реализации	26
6.3 Обеспечивающие операторы	26
7 Метод ветвей и границ	28
7.1 Оценка кардинальности	29
7.2 Оценка стоимости	29
8 Дифференциальный анализ физических планов	31
8.1 Предлагаемый метод	31

8.2	Поиск ближайшего плана	32
8.3	Область применения	33
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	34
	ПРИЛОЖЕНИЕ А	35

ВВЕДЕНИЕ

В эпоху стремительного роста объемов данных системы управления базами данных (СУБД) являются ключевой частью информационных систем. По данным аналитических исследований, объем мирового рынка СУБД ежегодно увеличивается, что обусловлено как расширением круга решаемых задач, так и усложнением структуры обрабатываемых данных и увеличением их объема. Реляционные базы данных, работающие с различными диалектами языка SQL, продолжают занимать доминирующее положение в сегменте обработки транзакций и аналитических запросов.

Одним из определяющих факторов конкурентоспособности между различными реализациями СУБД является качество оптимизатора запросов. Оптимизатор представляет собой компонент, ответственный за преобразование декларативного SQL-запроса в эффективный физический план исполнения. Именно от качества работы оптимизатора в значительной степени зависит производительность всей системы: выбор неоптимального плана исполнения может привести к увеличению времени выполнения запроса в десятки раз относительно оптимального случая.

Также оптимизатор запросов считается наиболее сложной составной частью любой СУБД. Задача нахождения оптимального плана является NP-полной в общем случае, что обуславливает необходимость применения эвристических методов и алгоритмов ограниченного перебора. Современные коммерческие и открытые системы используют различные архитектуры оптимизаторов, среди них наиболее популярны устоявшиеся и проверенные временем архитектуры Cascades и Volcano.

Архитектура Cascades используется в таких промышленных системах, как Microsoft SQL Server, Apache Orca, а также в ряде исследовательских проектов, включая Columbia и CockroachDB. Преимущества этой архитектуры заключаются в расширяемости, модульности, а также понятном и эффективном алгоритме поиска, допускающим параллельность исполнения.

Целью данной работы является разработка и реализация оптимизатора подмножества SQL-запросов на основе архитектуры Cascades, а также создание фреймворка для его итеративного улучшения путем сравнения с промышленными реализациями.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить существующие архитектуры оптимизаторов запросов, включая System R, Volcano и Cascades;
- формализовать подмножество SQL и соответствующее подмножество реляционной алгебры;
- реализовать структуру данных Memo и алгоритм поиска оптимального плана на основе архитектуры Cascades;
- реализовать набор правил трансформации и реализации для основных операторов реляционной алгебры;
- реализовать метод ветвей и границ для эффективного отсеечения неоптимальных планов;
- реализовать метод дифференциального анализа физических планов для автоматизированного поиска примеров неоптимальных планов.

1 Архитектура СУБД

Система управления базами данных представляет собой программный комплекс, обеспечивающий хранение, извлечение и модификацию структурированных данных. Несмотря на разнообразие существующих реализаций, архитектура большинства реляционных СУБД включает ряд типовых компонентов, взаимодействие которых обеспечивает выполнение пользовательских запросов.

К основным компонентам СУБД относятся (рисунок 1):

- транспортный уровень, принимающий клиентские подключения и запросы;
- синтаксический анализатор (парсер), выполняющий разбор текста SQL-запроса и построение синтаксического дерева;
- оптимизатор запросов, преобразующий логическое представление запроса в эффективный физический план исполнения;
- исполнитель, непосредственно реализующий физический план;
- менеджер хранения данных, управляющий размещением данных на физических носителях;
- менеджер транзакций и менеджер блокировок, обеспечивающие гарантии транзакций;
- менеджер страниц памяти, отвечающий за кэширование страниц данных в оперативной памяти.

1.1 Процесс исполнения SQL-запроса

Процесс исполнения SQL-запроса в реляционной СУБД проходит несколько последовательных этапов.

1. Синтаксический анализ — запроса проходит лексический и синтаксический разбор. Результатом данного этапа является синтаксическое дерево, узлы которого соответствуют конструкциям языка SQL: операторам SELECT, FROM, WHERE, JOIN, GROUP BY, ORDER BY и другим. На этом же

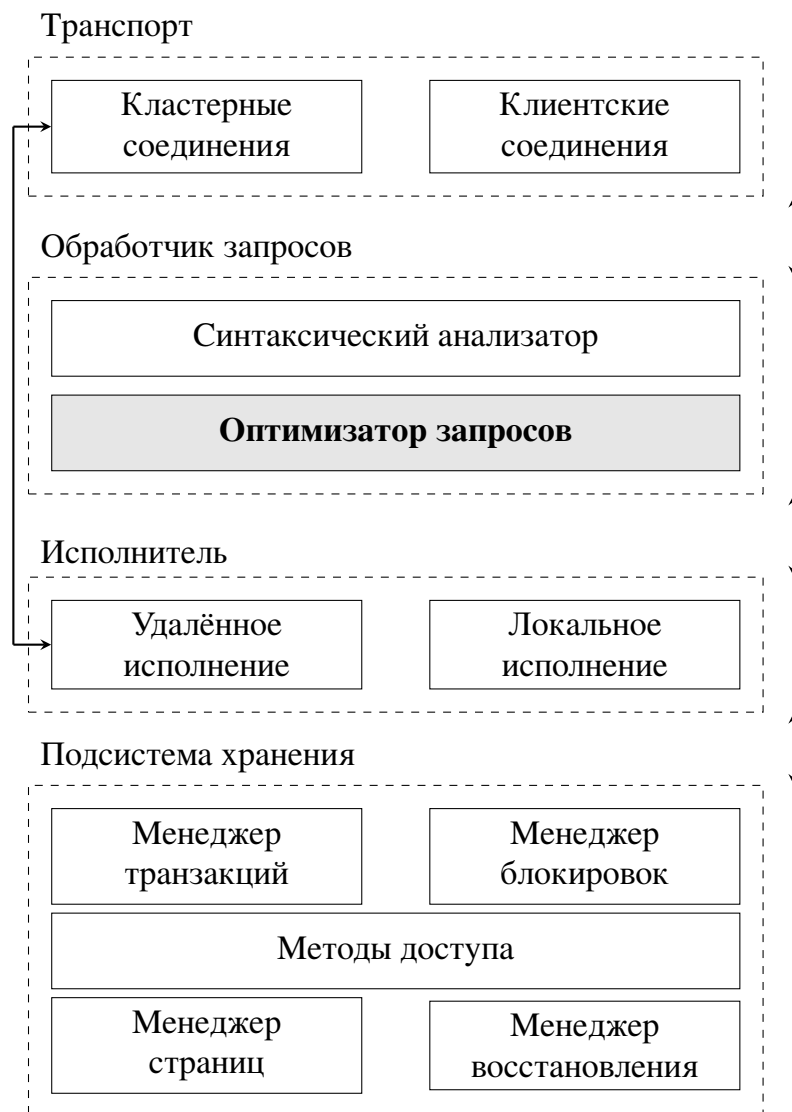


Рисунок 1 — Архитектура реляционной системы управления базами данных

этапе выполняется семантический анализ: проверяется существование указанных таблиц и столбцов, разрешаются имена объектов и определяются типы выражений.

2. Оптимизация — синтаксическое дерево преобразуется в логический план запроса, выраженный в терминах реляционной алгебры. Оптимизатор исследует пространство эквивалентных логических планов и для каждого из них рассматривает возможные физические реализации операторов. С помощью модели стоимости оптимизатор оценивает затраты на выпол-

нение каждого плана и выбирает план с наименьшей оценочной стоимостью. Результатом этого этапа является физический план исполнения. Под этим понимается дерево физических операторов с указанием конкретных алгоритмов соединения, методов доступа к данным и порядка операций.

3. Исполнение физического плана — движок выполнения реализует физический план, порождая потоки кортежей. Данные извлекаются из хранилища, обрабатываются операторами плана и формируют результирующий набор, возвращаемый пользователю.

2 Реляционная алгебра

Реляционная алгебра представляет собой формальный язык для описания операций над отношениями реляционной базы данных. В отличие от декларативного SQL, реляционная алгебра процедурной, так как она определяет конкретную последовательность операций, необходимых для получения результата.

Отношение определяется как конечное множество кортежей, каждый из которых представляет собой упорядоченный набор значений атрибутов:

$$R \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n).$$

2.1 Основные операторы реляционной алгебры

Среди основных операторов реляционной алгебры выделяют следующие.

1. Фильтрация $\sigma_p(R)$ — возвращает подмножество кортежей отношения R , удовлетворяющих предикату p . Например, $\sigma_{\text{age} > 30}(\text{Employee})$ вернет все кортежи из отношения `Employee`, для которых значение атрибута `age` больше 30.
2. Проекция $\pi_{A_1, \dots, A_k}(R)$ — формирует новое отношение, содержащее только перечисленные атрибуты.
3. Декартово произведение $R \times S$ — формирует отношение, каждый кортеж которого является конкатенацией кортежа из R и кортежа из S , при этом результат содержит $|R \times S| = |R| \cdot |S|$ элементов.
4. Соединение $R \bowtie_p S$ — комбинирует кортежи двух отношений на основании предиката p . Внутреннее соединение определяется как $R \bowtie_{R.a=S.b} S = \sigma_{R.a=S.b}(R \times S)$.
5. Объединение $R \cup S$ — возвращает все кортежи, принадлежащие хотя бы одному из двух совместимых по схеме отношений.
6. Разность $R - S$ — выбирает кортежи, принадлежащие R , но отсутствующие в S .

7. Агрегация $\gamma_{G,f}(R)$ — разбивает отношение R на непересекающиеся множества по набору атрибутов G и вычисляет агрегированные значения функции f (такие как COUNT, SUM, AVG, MIN, MAX) по каждому такому множеству.

2.2 Преобразование синтаксического дерева к реляционной алгебре

Полученное после лексического и синтаксического разбора абстрактное синтаксическое дерево обычно преобразовывают в форму операторов реляционной алгебры.

Это преобразование выполняется по фиксированному набору правил, отражающих семантику конструкций SQL:

- блок SELECT-FROM-WHERE превращается в проекцию над селекцией над декартовым произведением соединяемых таблиц;
- конструкция GROUP BY порождает оператор группировки;
- ORDER BY — запрос на требуемое физическое свойство сортированности;
- вложенные подзапросы — оператор Apply, который впоследствии может быть удалён процедурой декорреляции.

Данное преобразование необходимо по нескольким причинам. Во-первых, реляционная алгебра обладает формализованным набором правил эквивалентности, которые позволяют оптимизатору корректно преобразовывать план запроса без изменения семантики. Во-вторых, алгебраическое представление позволяет единообразно обрабатывать различные синтаксические формы SQL-запросов, порождающие одинаковые логические планы. Наконец, разделение на логические и физические операторы позволяет оптимизатору независимо исследовать порядок операций и алгоритмы их реализации.

3 Цель оптимизатора

Сформулируем задачу, которую решает оптимизатор: по запросу, представленному в форме дерева операторов реляционной алгебры, построить эквивалентный и оптимальный, с точки зрения необходимых для исполнения ресурсов, план исполнения. Итоговый план является деревом, вершины которого помечены конкретными указаниями на алгоритмы, реализующими один или несколько операторов реляционной алгебры. Эквивалентность означает, что итоговый физический план и исходный запрос при исполнении на любом наборе данных порождают одинаковые отношения с точностью до требуемых свойств, таких как сортировка.

Запрос в форме реляционной алгебры также называют логическим планом, а операторы реляционной алгебры — логическими операторами.

Для каждого логического оператора существует несколько возможных физических реализаций. Например, оператор соединения $\bowtie$ может быть реализован как соединение вложенными циклами, хеш-соединение или соединение слиянием. Аналогично, оператор доступа к данным может быть реализован как последовательное сканирование таблицы или сканирование индекса. Еще одним важным примером является эквивалентность сканирования индекса с предикатом и двух логических операторов, примененных последовательно: доступ к данным и фильтрация по тому же предикату.

Эквивалентность планов обеспечивается свойствами реляционной алгебры.

Пример 1. *Коммутативность соединения:*

$$R \bowtie S \equiv S \bowtie R.$$

Пример 2. *Ассоциативность соединения:*

$$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T).$$

Оптимальность запроса оценивается с помощью функции стоимости $\text{cost}: \mathcal{P}_L \rightarrow \mathbb{R}_{\geq 0}$, где $\mathcal{P}_L$ — множество эквивалентных физических планов. Это

отображение оценивает затраты на исполнение плана исходя из конкретных реализаций алгоритмов, а также основываясь на эвристиках и статистике о данных в отношениях. Другими словами, цель оптимизатора состоит в том, чтобы для входного запроса найти план $P^* \in \mathcal{P}_L$, минимизирующий cost:

$$P^* = \arg \min_{P \in \mathcal{P}_L} \text{cost}(P).$$

Одной из подзадач рамках построения оптимального плана является определение порядка выполнения соединений. Для соединения n отношений количество возможных порядков их выполнения выражается как $O(n! \cdot C_{n-1}) = O(\frac{(2n-2)!}{(n-1)!})$ [1], где C_k — k -е число Каталана. Таким образом, пространство поиска растет экспоненциально с увеличением числа соединяемых таблиц.

Поскольку полный перебор $\mathcal{P}_L$ для достаточно больших n невозможен, на практике речь идёт о поиске плана с приемлемо малой стоимостью в определенном подмножестве пространства поиска. Именно эта задача и определяет архитектуру современных оптимизаторов: расширяемое описание пространства поиска через правила преобразования, эффективный алгоритм его исследования и стоимостная модель, позволяющая отсекаать заведомо неоптимальные ветви.

Выбор плана исполнения запроса может оказывать огромное влияние на объем требуемых вычислительных ресурсов. В качестве примера оценим количество чтений страниц с диска для различных физических операторов. Пусть имеется три отношения: таблица `customer` с $|C| = 10^5$ строк, таблица `orders` с $|O| = 10^7$ строк и таблица `lineitem` с $|L| = 5 \cdot 10^7$ строк. Размер страницы — 8 КБ. В одну страницу помещается 100 строк `customer`, 80 строк `orders` и 50 строк `lineitem`. Соответственно, общее число страниц составляет $P_C = 10^3$, $P_O = 1,25 \cdot 10^5$, $P_L = 10^6$.

Для запроса из листинга 1 рассмотрим два альтернативных плана (рисунок 2):

- План P_1 : $(\text{customer} \bowtie \text{orders}) \bowtie \text{lineitem}$ с реализацией обоих соединений через Hash Join, для чего сначала полностью считываются

Листинг 1: Соединение customer, orders и lineitem.

```
SELECT *
FROM customer c
JOIN orders o ON c.c_custkey = o.o_custkey
JOIN lineitem l ON o.o_orderkey = l.l_orderkey
```

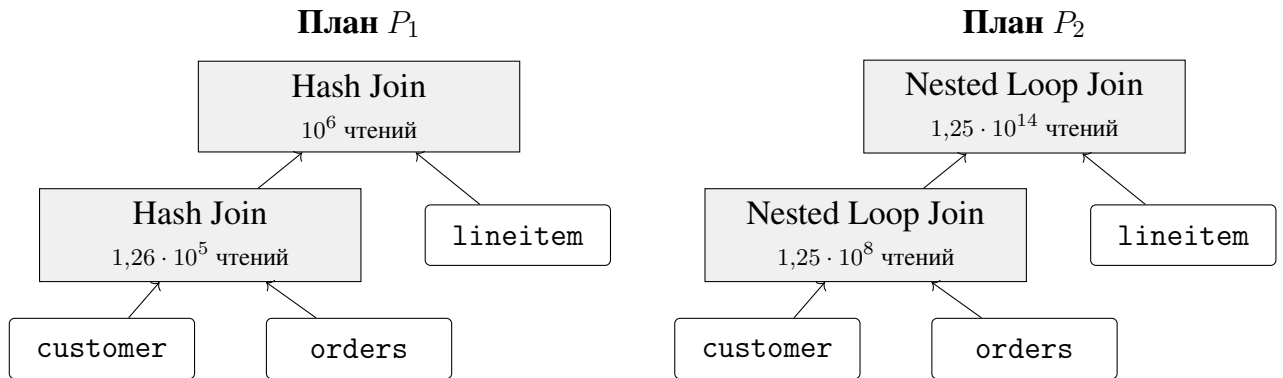


Рисунок 2 — Сравнение планов для запроса из листинга 1.

$P_C + P_O = 1,26 \cdot 10^5$ страниц для построения хеш-таблицы, после чего промежуточный результат размером 10^7 строк соединяется с lineitem, требуя считывания ещё $P_L = 10^6$ страниц. В сумме — порядка $1,126 \cdot 10^6$ операций чтения с диска.

- План P_2 : тот же запрос, но реализованный как соединение с помощью вложенных циклов без использования индексов: для каждой строки customer полностью читается таблица orders, для каждой результирующей пары — lineitem. Совокупный объём ввода-вывода оценивается величиной $P_C \cdot P_O \cdot P_L \approx 1,25 \cdot 10^{14}$ операций чтения, что на восемь порядков превышает P_1 .

На еще одном примере запроса из листинга 2 рассмотрим влияние логических преобразований. Отношение Emp содержит 10,000 кортежей, что эквивалентно 1,000 страницам. Dept включает 500 записей и 50 страниц соответственно. Атрибут Emp.did является внешним ключом к Dept.did, поэтому каждому кортежу

Листинг 2: Пример запроса с проекцией, фильтрацией и соединением.

```
SELECT DISTINCT ename
FROM Emp E JOIN Dept D ON E.did = D.did
WHERE D.dname = 'Toy'
```

из Dept соответствует $10,000/500 = 20$ из Emp. Пусть на Emp.did и Dept.dname построены индексы, а в промежуточную страницу помещается 5 кортежей.

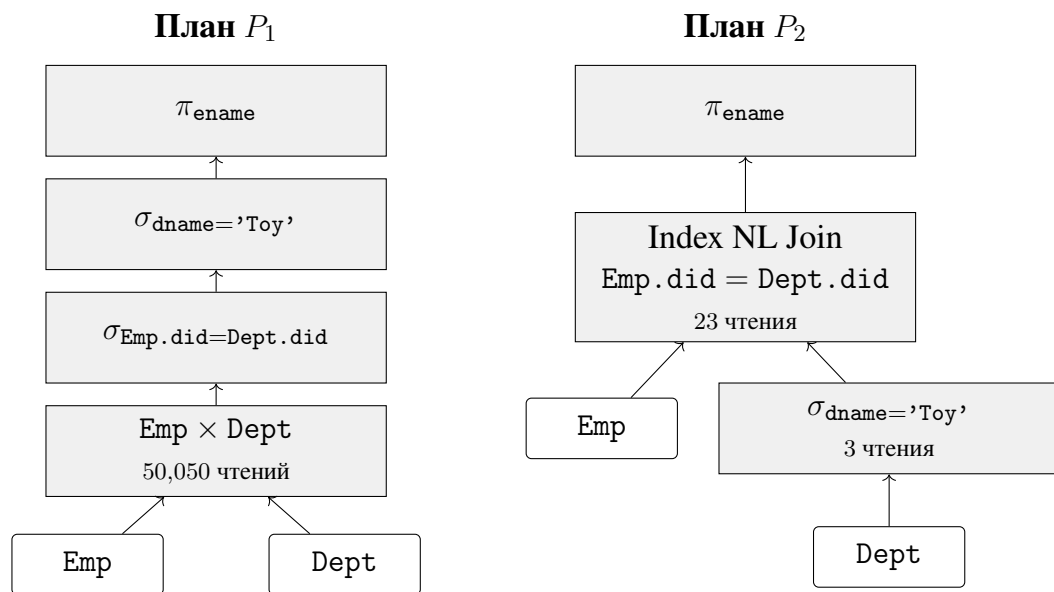


Рисунок 3 — Сравнение планов для запроса из листинга 2.

Рассмотрим два плана (рисунок 3):

- План P_1 соответствует прямому преобразованию запроса в реляционную алгебру. Если рассматривать стоимость исполнения такого плана в модели итераторов, то каждый оператор запрашивает у дочернего следующий кортеж через вызов `next()`, поэтому σ и π работают в режиме конвейера и не инициируют обращений к диску. Вся стоимость ввода-вывода сосредоточена в операторе $\times$, реализованном через вложенные циклы с Dept в роли внешней таблицы: Dept читается один раз, а Emp

перечитывается полностью для каждой страницы Dept:

$$50 + 50 \times 1,000 = 50,050 \text{ чтений.}$$

- План P_2 является оптимизированным вариантом, где применено правило проталкивания предиката и декартово произведение заменено на соединение по предикату. Причем для доступа к таблице Dept применяется сканирование по индексу, что дает всего 3 чтения с диска. Соединение реализуется через вложенные циклы с использованием индекса, что в итоге дает:

$$3 + (3 + 20) = 26 \text{ чтений.}$$

Оба приведенных примера наглядно демонстрируют, что оптимизатор отвечает не за доли процентов производительности, а за принципиальную возможность исполнения сложных аналитических запросов за приемлемое время. В первом случае разница между физическими планами составляет восемь порядков: если P_1 завершается за секунды, то наивный P_2 потребует десятков лет. Во втором случае даже единственное логическое преобразование, в виде протаскивания предиката, снижает число чтений с диска в 1,900 раз. Именно поэтому качественный оптимизатор является конкурентным преимуществом и одной из самых важных частей СУБД.

4 Обзор архитектур оптимизаторов

Рассмотрим наиболее популярные архитектуры оптимизаторов.

4.1 System R

Оптимизатор System R, разработанный в IBM Research в конце 1970-х годов [2], стал первым стоимостным оптимизатором запросов и заложил фундаментальные принципы, используемые во всех последующих системах.

Архитектуру, аналогичную System R, реализуют PostgreSQL и ранние версии IBM DB2.

Оптимизатор, построенный по архитектуре System R, разбивает запрос на блоки, каждый из которых оптимизируется индивидуально. Блок запроса состоит из конструкции SELECT, конструкции FROM, ссылающейся на одну или несколько таблиц, и дерева предикатов WHERE. Несколько блоков появляется в случае запроса с вложенными подзапросами.

Для каждого блока оптимизатор выполняет два основных действия. Во-первых, для запросов к одному отношению выбирается наилучший метод доступа на основании стоимостной модели. Во-вторых, для запросов с несколькими отношениями определяется оптимальный порядок соединений методом восходящего динамического программирования: сначала выбираются оптимальные планы доступа к отдельным таблицам, затем перебираются двухтабличные соединения, трехтабличные и так далее.

Стоимость плана вычисляется по формуле $COST = PageFetches + w \cdot RSICalls$, учитывающей ожидаемый объем операций ввода-вывода и потребление ресурсов процессора, где w — весовой коэффициент.

Важным нововведением System R стало понятие интересных порядков. Стоимость плана зависит не только от непосредственных затрат на выполнение оператора, но и от того, обеспечивает ли он упорядоченность результата, требуемую вышестоящими операторами. Например, использование соединения слия-

нием вместо хеш-соединения может быть выгодно, если результат должен быть отсортирован для последующей операции ORDER BY.

К ограничениям архитектуры System R относятся:

- рассмотрение только левосторонних деревьев соединений;
- независимая оптимизация вложенных подзапросов;
- невозможность гибкого расширения набора правил преобразования, что необходимо при добавлении новых операторов языка SQL.

Эти ограничения послужили мотивацией для создания расширяемых архитектур оптимизаторов.

4.2 Volcano

Volcano — расширяемый, основанный на правилах, стоимостной оптимизатор запросов, разработанный Грефе в 1993 году [3]. Volcano ввел ряд концепций, ставших основой для последующих расширяемых оптимизаторов: физические свойства выражений, обеспечивающие правила, структуру данных Мемо и нисходящий алгоритм поиска на основе динамического программирования.

Volcano разделяет поиск на две фазы:

- фаза генерации — оптимизатор применяет правила трансформации для порождения всех эквивалентных логических выражений. Применяется рекурсивный нисходящий обход дерева: для каждого выражения сначала исследуются дочерние выражения, затем применяются правила трансформации к текущему выражению. Все порожденные выражения кэшируются в структуре данных Мемо для предотвращения дублирования.
- фаза стоимостного анализа — по завершении генерации всех эквивалентных логических выражений оптимизатор ищет наилучший физический план. Для каждого логического выражения применяются правила реализации, порождающие физические операторы. Стоимость каждого физического плана оценивается рекурсивно, и сохраняется наилучший найденный план.

Поиск осуществляется нисходящим динамическим программированием с использованием стратегии ветвей и границ: текущая верхняя граница стоимости передается при рекурсивных вызовах и используется для отсеечения заведомо неоптимальных планов.

Основным недостатком Volcano является полное раскрытие всех классов эквивалентности на фазе генерации до начала стоимостного анализа, что приводит к избыточным вычислениям.

5 Архитектура Cascades

Cascades — это расширяемая архитектура оптимизатора запросов, являющаяся логическим продолжением идеи Volcano [4].

Основными объектами Cascades являются:

- выражение — дерево или фрагмент дерева, корнем которого является логический либо физический оператор;
- групповое выражение — выражение, дочерние узлы которого являются ссылками на группы Мемо;
- группа — множество логически эквивалентных выражений, возвращающих один и тот же результат;
- правило трансформации — правило, преобразующее логическое выражение в логически эквивалентное выражение;
- правило реализации — правило, преобразующее логический оператор в физический оператор;
- физическое свойство — требование относительно порядка, распределения, разбиения или уникальности результата;
- задача — элементарная единица работы оптимизатора, например исследование группы, применение правила или оптимизация выражения под заданное свойство.

Важным принципом данной архитектуры является то, что правила являются объектами первого класса. Они определяются условиями применимости и собственно алгоритмом применения. Предполагается, что система легко расширяется добавлением новых правил, потому что все правила наследуются от общего интерфейса и имеют одинаковую структуру.

5.1 Группы эквивалентности и выражения

Группа в Мемо представляет собой множество выражений, которые эквивалентны с точки зрения логического результата. Например, для отношений A , B и C выражения

$$A \bowtie (B \bowtie C); \quad (A \bowtie B) \bowtie C$$

при выполнении условий ассоциативности соединения принадлежат одной группе, поскольку возвращают одинаковый результат. Причем физические реализации этих выражений могут отличаться, а стоимость может зависеть от кардинальности промежуточных результатов.

Групповое выражение хранит оператор и ссылки на дочерние группы. Например, выражение $Join(G_1, G_2)$ не указывает конкретное дерево для левого и правого входов, а ссылается на группы G_1 и G_2 , каждая из которых может содержать множество альтернатив. Поэтому одно групповое выражение кодирует комбинацию многих конкретных деревьев, храня эту информацию компактно.

5.2 Структура Мемо

Мемо является основной структурой данных Cascades. Оно выполняет сразу несколько функций: устраняет дублирующиеся выражения, хранит альтернативы, фиксирует результаты оптимизации под разными физическими свойствами и обеспечивает точку синхронизации для задач поиска.

Группа в Мемо включает следующую информацию:

- уникальный идентификатор группы;
- список логических выражений;
- список физических выражений;
- логические свойства группы;
- статистику и оценку кардинальности;
- состояние исследования группы.

Выражение, в свою очередь, содержит сам оператор, массив идентификаторов дочерних групп. Для физического выражения дополнительно могут храниться предоставляемые физические свойства, например сортировка выходного потока, и рассчитанная стоимость исполнения этого оператора.

Листинг 3: Пример запроса.

```
SELECT * FROM A JOIN B ON p_1 JOIN C ON p_2;
```

На рисунке 4 показан пример Мемо для запроса из листинга 3. Например, группа G_5 представляет результат соединения трех отношений, в ней находятся два логически эквивалентных выражения с разной ассоциацией соединений и четыре физические реализации для них. Входными данными для операторов являются группы группы, поэтому общие подвыражения не дублируются.

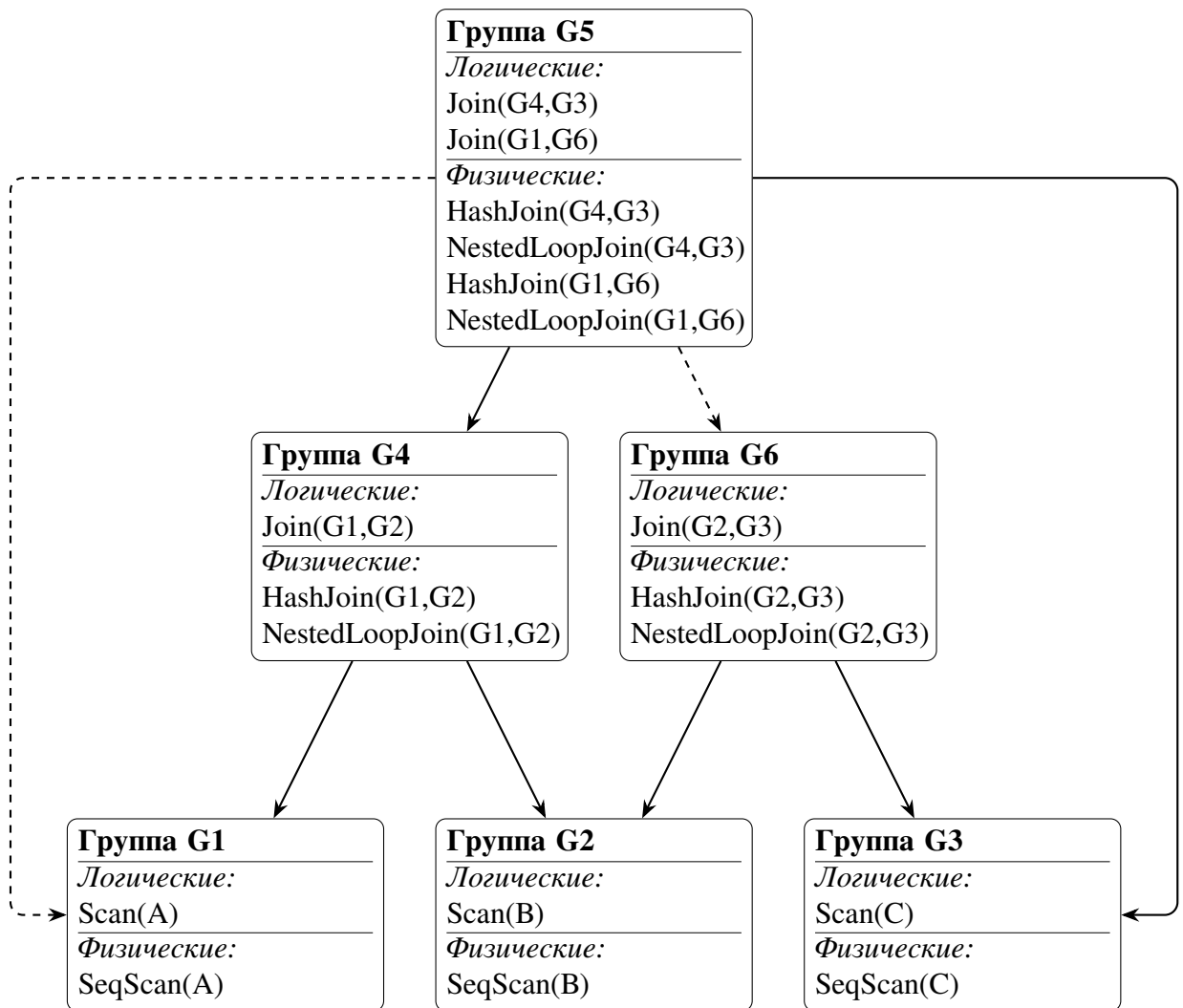


Рисунок 4 — Пример структуры Мемо.

При добавлении нового выражения Мемо выполняет проверку на дубликаты. Если выражение уже существует в некоторой группе, оно не добавляется повторно. Если алгоритм в процессе работы обнаруживает эквивалентность двух ранее различных групп, то они могут быть объединены. При этом требуется обновление ссылок и состояния правил.

5.3 Алгоритм поиска

Cascades организует оптимизацию как выполнение набора задач. Задача может исследовать группу, исследовать выражение, применить правило, реализовать логический оператор, оптимизировать дочернюю группу под заданное физическое свойство или построить вспомогательный оператор для обеспечения свойства. Такой подход позволяет гибко управлять порядком работы и откладывать дорогостоящие действия до тех пор, пока они не станут необходимыми, а также разбивать поиск на несколько этапов и распараллеливать исполнение независимых веток. При этом однопоточные реализации хранят задачи в обычной очереди.

На рисунке 5 показана схема взаимодействия задач в оптимизаторе, а на листингах 1-3 приведен псевдокод алгоритма поиска в архитектуре Cascades [5].

Алгоритм реализуется в виде управляемого стеком поиска в пространстве эквивалентных планов: сначала запускается оптимизация корневой группы, `OptimizeGroup` в свою очередь исследует группу, а затем оптимизирует каждое ее логическое выражение. Исследование выполняется через `ExploreGroup` и `ExploreExpression`, которые сохраняют в Мемо новые логические выражения и группы с помощью правил преобразования. После этого `OptimizeExpression` применяет правила реализации, превращая логические выражения в физические. Для каждого физического варианта `ApplyImplementation` проверяет локальную стоимость и передает план в `OptimizeInputs`, где последовательно оптимизируются дочерние группы, накапливается полная стоимость и обновляется информация о лучшем плане. Параметр `limit` используется для отсеечения заведомо дорогих вариантов.

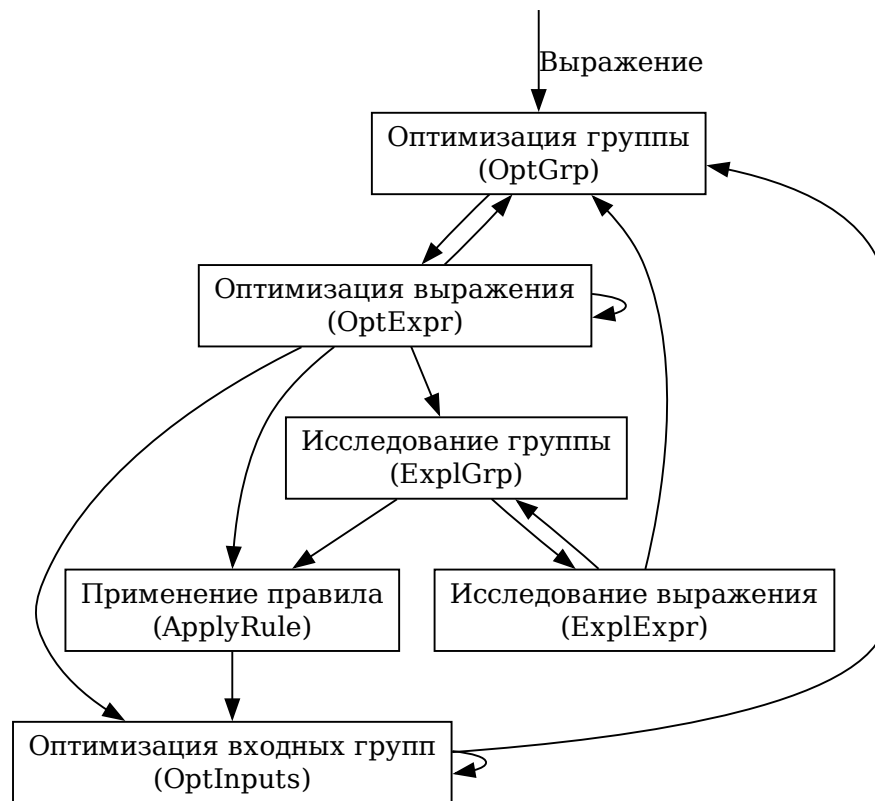


Рисунок 5 — Схема взаимодействия задач в оптимизаторе Cascades

6 Правила трансформации и реализации

Правило в оптимизаторе Cascades состоит из образца, условия применимости и алгоритма применения. Образец описывает форму логического выражения, к которому может быть применено правило. Условие применимости проверяет дополнительные требования: отсутствие внешних ссылок, тип соединения, сохранение семантики NULL, совместимость свойств и другие ограничения.

Формально правило можно представить как отображение:

$$r : pattern(e) \wedge condition(e) \rightarrow substitute(e),$$

где e — исходное выражение, $pattern$ — структурный образец, $condition$ — предикат применимости, а $substitute$ — выражение-замена. Для правил трансформации результатом применения является новое логическое выражение в группе, а для правил реализации — физическое выражение в той же группе.

6.1 Правила трансформации

Наиболее важные правила трансформации направлены на уменьшение промежуточных кардинальностей, удаление лишних атрибутов и исследование новых порядков выполнения соединения.

Правило проталкивания предиката перемещает выборку ниже оператора соединения, если предикат обращается только к атрибутам одного входа. Для внутреннего соединения это правило можно записать следующим образом:

$$\sigma_p(R \bowtie_q S) \Rightarrow (\sigma_p(R)) \bowtie_q S, \quad attrs(p) \subseteq attrs(R).$$

Условие активации требует, чтобы предикат не зависел от атрибутов второго входа и чтобы его перемещение не меняло семантику внешнего соединения. Для левого внешнего соединения проталкивание предиката в правый вход может быть некорректным, если предикат фильтрует строки, которые должны быть дополнены значениями NULL.

Правило проталкивания проекции уменьшает ширину промежуточных кортежей. Если родительские операторы используют только часть атрибутов входа, оптимизатор может удалить ненужные столбцы раньше, снизив объем памяти и стоимость передачи данных. Условие активации требует сохранить все атрибуты, необходимые для предикатов, соединений, группировок и последующих вычисляемых выражений.

Правила коммутативности и ассоциативности соединений исследуют пространство возможных порядков применения соединения:

$$R \bowtie S \Rightarrow S \bowtie R,$$

$$R \bowtie (S \bowtie T) \Rightarrow (R \bowtie S) \bowtie T.$$

Эти правила безопасны для внутренних соединений при корректном переносе предикатов, однако для внешних соединений их условия применимости значительно сложнее. Поэтому промышленные системы часто используют специальные алгоритмы, учитывающие зависимости в предикатах с помощью представлений в виде графов.

Правило объединения выборок заменяет два последовательных фильтра одним с конъюнкцией предикатов:

$$\sigma_p(\sigma_q(R)) \Rightarrow \sigma_{p \wedge q}(R).$$

Такое преобразование упрощает дальнейший анализ селективности и позволяет сопоставить составной предикат с составным индексом. Обратное правило также может быть полезно, если оптимизатору нужно разнести конъюнкты по разным ветвям плана.

Правило удаления лишнего соединения применимо, если соединяемая таблица не вносит атрибутов в результат, а ограничения ключей и внешних ключей гарантируют, что соединение не изменяет множество строк. Например, соединение таблицы заказов с таблицей клиентов может быть удалено, если результат

использует только атрибуты заказов, а ограничение внешнего ключа гарантирует существование соответствующего клиента.

TODO: вообще плохо написан раздел про трансформации. Расширить и более четко описать

6.2 Правила реализации

Правила реализации переводят логические операторы в физические. Например, для логического оператора чтения таблицы могут быть применены правила:

- $\text{LogicalGet} \Rightarrow \text{SeqScan}$;
- $\text{LogicalGet} \Rightarrow \text{IndexScan}$, если имеется индекс, совместимый с предикатом или требуемым порядком.

Для логического соединения могут быть созданы физические операторы `NestedLoopJoin`, `HashJoin` и `MergeJoin`. Хеш-соединение обычно требует предиката равенства по ключам, поскольку именно по ним строится хеш-таблица. Сортировочное соединение требует, чтобы оба входа были отсортированы по ключам соединения, либо чтобы оптимизатор мог добавить сортировки как обеспечивающие операторы. Вложенные циклы применимы всегда, но стоимость этого оператора может быть высокой без доступа к внутреннему отношению с использованием индекса.

Правила реализации агрегации выбирают между хеш-агрегацией и потоковой агрегацией. Хеш-агрегация не требует предварительного порядка, но потребляет память пропорционально числу групп. Потокковая агрегация эффективна при уже отсортированном входе, поскольку каждая группа обрабатывается последовательно. Если требуемый порядок нужен также родительскому оператору, потоковая агрегация может оказаться выгодной даже учитывая затраты на сортировку.

6.3 Обеспечивающие операторы

Физические свойства в `Cascades` обрабатываются через требуемые и предоставляемые свойства. Если родительский оператор требует вход, отсортирован-

ный по атрибуту a , а выбранный дочерний план такого порядка не предоставляет, оптимизатор может добавить обеспечивающий оператор Sort.

Обеспечивающие операторы не изменяют логический результат, но меняют физические свойства и увеличивают стоимость. Поэтому они рассматриваются наравне с другими физическими альтернативами. Например, план с хеш-соединением и последующей сортировкой может конкурировать с планом на основе соединения слиянием, которое сохраняет порядок результата.

7 Метод ветвей и границ

Метод ветвей и границ используется для сокращения пространства поиска и, как следствие, неасимптотического улучшения скорости работы алгоритма. В контексте Cascades ветвями являются альтернативные выражения и комбинации физических реализаций дочерних групп, а границей является текущая лучшая стоимость плана для заданной группы и требуемых физических свойств. Если нижняя оценка стоимости частично построенного плана уже превышает текущую лучшую, дальнейшее исследование по этой ветке не имеет смысла.

Пусть для группы G и требуемого свойства P уже найден план стоимости C^* . Рассматривается физическое выражение e , имеющее локальную стоимость $C_{local}(e)$ и дочерние группы $G_1, \dots, G_k$. Если даже минимально возможная оценка дочерних групп приводит к неравенству

$$C_{local}(e) + \sum_{i=1}^k LB(G_i, P_i) \geq C^*,$$

то выражение e может быть отсечено. Здесь LB обозначает нижнюю границу стоимости оптимизации дочерней группы под требуемым свойством P_i . Чем точнее нижняя граница, тем эффективнее отсечение.

На практике в качестве $LB(G_i, P_i)$ принимается лучшая стоимость, найденная для пары (G_i, P_i) на текущий момент работы алгоритма. Поскольку оптимизация группы является рекурсивным процессом, к моменту отсечения для (G_i, P_i) уже могут быть найдены некоторые планы, и их минимальная стоимость служит нижней границей. Если же группа G_i под свойством P_i ещё не рассматривалась, используется тривиальная граница $LB = 0$, или, при условии наличия минимальной стоимости на кортеж, можно принимать $LB(G_i, P_i) = |G_i| \cdot \min_{cost_per_row}$.

Требуемые свойства дочерних групп P_i определяются оператором выражения e совместно с требованием родителя P . Каждый физический оператор описывает, какие свойства он ожидает от своих входов: например, оператор сортирующего слияния требует упорядоченности обоих входов по соответствующим ключам, тогда как хеш-соединение не предъявляет требований к порядку. Таким

образом, при спуске по дереву поиска каждый оператор транслирует глобальное требование P в конкретные требования $P_1, \dots, P_k$ к своим дочерним группам.

7.1 Оценка кардинальности

Оценка кардинальности является процессом предсказания числа кортежей, возвращаемых оператором.

Для выборки используется селективность предиката $sel(p)$, после чего кардинальность результата оценивается как

$$|\sigma_p(R)| = |R| \cdot sel(p).$$

Для соединения на основе предиката равенства независимых атрибутов часто применяется эвристика:

$$|R \bowtie_{R.a=S.b} S| \approx \frac{|R| \cdot |S|}{\max(NDV(R.a), NDV(S.b))},$$

где NDV — число различных значений атрибута. Такая оценка удобна, но может быть грубой, если данные имеют корреляции, сильный перекося или сложные зависимости между атрибутами. Экспериментальные исследования показывают, что ошибки кардинальности часто являются одной из основных причин выбора неудачных планов [6].

Для повышения качества оценок используются гистограммы, статистики по многоколоночным ключам, сведения об уникальности, функциональные зависимости, ограничения на схему, а в современных исследованиях также модели машинного обучения.

7.2 Оценка стоимости

Функция стоимости сопоставляет физическому плану численную оценку затрат. В простейшем случае стоимость можно представить как взвешенную сумму

операций чтения страниц, операций записи, вычислительных операций и сетевой передачи:

$$Cost(p) = w_{io} \cdot IO(p) + w_{cpu} \cdot CPU(p) + w_{mem} \cdot MEM(p) + w_{net} \cdot NET(p).$$

Коэффициенты w_{io} , w_{cpu} , w_{mem} и w_{net} зависят от конкретной аппаратной конфигурации.

Стоимость физического выражения обычно вычисляется рекурсивно:

$$Cost(node) = LocalCost(node) + \sum_{child \in children(node)} Cost(child).$$

При этом локальная стоимость зависит от кардинальности входов и физических свойств, а также конкретного физического оператора.

8 Дифференциальный анализ физических планов

Множество правил трансформации и реализации, используемых в оптимизаторах запросов, изучено довольно хорошо и описано в академической литературе. Вместе с тем условия активации правил существенно различаются от системы к системе. Многие реализации оптимизаторов упускают возможности для оптимизации из-за слишком строгих условий активации правил или отсутствия определенных правил в их наборе.

Данная проблема особенно актуальна при разработке нового оптимизатора: после реализации базового набора правил возникает вопрос о полноте и корректности этого набора по сравнению с устоявшимися и проверенными временем промышленными системами. Такие системы аккумулировали обратную связь от пользователей и уточняли условия активации своих правил в течение многих лет. Не имея такого преимущества сложно добиться отличного качества построенных планов.

8.1 Предлагаемый метод

Для поиска несоответствий между поведением реализуемой системы и внешней эталонной системы предлагается применить дифференциальный анализ планов.

Первым шагом нужно сформировать набор данных состоящий из схем таблиц и данных. Далее, по заданным схемам генерируются случайные SQL-запросы различной структуры и сложности. Такой подход исключает тривиальные планы и приближает условия работы оптимизатора к реальным нагрузкам при дальнейшем построении планов запросов.

Следующим шагом сгенерированные запросы транпилируются в диалекты SQL целевых промышленных СУБД: Oracle, Microsoft SQL Server, PostgreSQL и т.д. Затем для получения физических планов выполняется аналог EXPLAIN ANALYZE.

Полученные физические планы промышленных СУБД конвертируются в формат сериализованного плана разрабатываемого оптимизатора. Для этого необходимо создать набор отображений между физическими операторами различных систем. Например, Hash Match в SQL Server отображается в HashJoin в разрабатываемой системе, Clustered Index Scan — в IndexScan и так далее.

Корректность конвертации плана можно проверить путем исполнения исходного плана на внешней системе и полученного плана на разрабатываемой системе и сравнения полученных результатов.

Конвертированные планы сравниваются с планами, порождаемыми разрабатываемым оптимизатором на исходных запросах, с целью определить, мог ли существующий набор правил разрабатываемого оптимизатора породить план, эквивалентный плану промышленной системы. Для корректности сравнения необходимо полное исследование пространства эквивалентных планов. Если разрабатываемая система не может породить “эталонный” план, то условия активации или набор правил требует уточнения. Определение требуемых изменений лежит на разработчике системы, поэтому найденный запрос и примеры планов сохраняются для последующего анализа. В дальнейшем, полученные после конвертации “эталонные” планы можно использовать как тесты для оптимизатора, которые будут успешно выполняться после уточнения соответствующих правил активации.

8.2 Поиск ближайшего плана

В случае невозможности построить “эталонный” план разрабатываемым оптимизатором, предлагается искать ближайший план, т.е. план из пространства поиска разрабатываемого оптимизатора, максимально близкий к плану промышленной системы. При этом метрика расстояния между планами может быть определена как расстояние редактирования деревьев.

Анализ ближайшего плана может помочь упростить выявление трансформаций, которые разрабатываемый план по какой-то причине не применил.

8.3 Область применения

Предлагаемый метод дифференциального анализа применим не только для сравнения с проприетарными системами, но и с СУБД с открытым исходным кодом, позволяя уточнять условия применения правил.

Систематическое применение этого подхода позволяет построить итеративный процесс совершенствования оптимизатора:

- генерация запросов;
- запуск и сбор планов;
- приведение к единому виду, сравнение и выявление расхождений;
- добавление или корректировка правил;
- повторное сравнение.

Таким образом, дифференциальный анализ обеспечивает управляемое и измеримое улучшение качества оптимизатора и позволяет новым или открытым оптимизаторам быстрее достигать качества устоявшихся промышленных оптимизаторов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Introduction to the Join Ordering Problem. — URL: <https://www.querifylabs.com/blog/introduction-to-the-join-ordering-problem> ; (дата обращения: 01.05.2026).
2. Access path selection in a relational database management system / P. G. Selinger [и др.] // Proceedings of the 1979 ACM SIGMOD International Conference on Management of Data. — 1979. — С. 23—34. — (SIGMOD '79). — DOI: 10.1145/582095.582099. — (дата обращения: 01.05.2026).
3. Graefe G., McKenna W. J. The Volcano Optimizer Generator: Extensibility and Efficient Search // Proceedings of IEEE 9th International Conference on Data Engineering. — 1993. — С. 209—218. — DOI: 10.1109/ICDE.1993.344061. — (дата обращения: 01.05.2026).
4. Graefe G. The Cascades Framework for Query Optimization // IEEE Data Engineering Bulletin. — 1995. — Т. 18, № 3. — С. 19—29. — (дата обращения: 01.05.2026).
5. Ding B., Narasayya V., Chaudhuri S. Extensible Query Optimizers in Practice // Foundations and Trends in Databases. — 2024. — Т. 14, № 3/4. — С. 186—402. — DOI: 10.1561/19000000077. — (дата обращения: 01.05.2026).
6. How good are query optimizers, really? / V. Leis [и др.] // Proceedings of the VLDB Endowment. — 2015. — Т. 9, № 3. — С. 204—215. — (дата обращения: 01.05.2026).

ПРИЛОЖЕНИЕ А

Листинг 1 Алгоритм поиска (часть 1).

```
1: procedure Optimize(root)
2:   Push task OptimizeGroup(root.group,  $\perp$ ) onto task stack
3:   while task stack is not empty do
4:     Pop and execute next task
5:   return best_plan[root.group]
6: procedure OptimizeGroup(group, limit)
7:   if best_cost[group] exists and (limit =  $\perp$  or best_cost[group] < limit)
   then
8:     return
9:   if group is not explored then
10:    Push task OptimizeGroup(group, limit) onto task stack
11:    Push task ExploreGroup(group, limit) onto task stack
12:    return
13:   for each logical expression expr  $\in$  group do
14:     Push task OptimizeExpression(expr, limit) onto task stack
15: procedure ExploreGroup(group, limit)
16:   Mark group as explored
17:   for each logical expression expr  $\in$  group do
18:     Push task ExploreExpression(expr, limit) onto task stack
```

Листинг 2 Алгоритм поиска (часть 2).

```
1: procedure ExploreExpression(expr, limit)
2:   for each applicable transformation rule r do
3:     Push task ApplyTransformation(r, expr, limit) onto task stack
4:   for each child group child  $\in$  Children(expr) do
5:     if child is not explored then
6:       Push task ExploreGroup(child, limit) onto task stack
7: procedure OptimizeExpression(expr, limit)
8:   for each applicable implementation rule r do
9:     Push task ApplyImplementation(r, expr, limit) onto task stack
10:  for each child group child  $\in$  Children(expr) do
11:    if child is not explored then
12:      Push task ExploreGroup(child, limit) onto task stack
13: procedure ApplyTransformation(rule, expr, limit)
14:   new_expr  $\leftarrow$  Apply(rule, expr)
15:   Push task ExploreExpression(new_expr, limit) onto task stack
16: procedure ApplyImplementation(rule, expr, limit)
17:   phys  $\leftarrow$  Apply(rule, expr)
18:   lc  $\leftarrow$  CalcLocalCost(phys)
19:   if limit  $\neq \perp$  and lc  $\geq$  limit then
20:     return
21:   child_limit  $\leftarrow$  (limit =  $\perp$ ) ?  $\perp$  : limit - lc
22:   Push task OptimizeInputs(phys, child_limit, 0) onto task stack
```

Листинг 3 Алгоритм поиска (часть 3).

```
1: procedure OptimizeInputs(expr, limit, i)
2:   children  $\leftarrow$  Children(expr)
3:   if  $i \geq |children|$  then
4:     total  $\leftarrow$  local_cost[expr] + accum_child_cost[expr]
5:     if best_cost[expr.group] does not exist or total < best_cost[expr.group]
       then
6:       best_cost[expr.group]  $\leftarrow$  total
7:       best_plan[expr.group]  $\leftarrow$  expr
8:     return
9:   child  $\leftarrow$  children[i]
10:  Push task: if best_cost[child] exists then accumulate child cost and push
    OptimizeInputs(expr, ..., i + 1)
11:  Push task OptimizeGroup(child, limit) onto task stack
```
